

Uncovering Software-Based Power Side-Channel Attacks on Apple M1/M2 Systems

Nikhil Chawla, Chen Liu, Abhishek Chakraborty, Igor Chervatyuk, Ke Sun, Thaís Moreira Hamasaki, Henrique Kawakami

{nikhil.chawla, chen.l.liu, abhishek.l.chakraborty, igor.chervatyuk, ke.sun, thais.moreira.hamasaki, henrique.kawakami}@intel.com
Intel Corporation, USA

Abstract

Traditionally, power side-channel analysis requires physical access to the target device, as well as specialized devices to measure the power consumption with enough precision. Recently research has shown that on x86 platforms, on-chip power meter capabilities exposed to a software interface might be used for power side-channel attacks without physical access. In this paper, we show that such software-based power side-channel attack is also applicable on Apple silicon (e.g., M1/M2 platforms), exploiting the System Management Controller (SMC) and its power-related keys, which provides access to the on-chip power meters through a software interface to user space software. We observed data-dependent power consumption reporting from such SMC keys and analyzed the correlations between the power consumption and the processed data. Our work also demonstrated how an unprivileged user mode application successfully recovers bytes from an AES encryption key from a cryptographic service supported by a kernel mode driver in macOS. We have also studied the feasibility of performing frequency throttling side-channel attack on Apple silicon. Furthermore, we discuss the impact of software-based power side-channels in the industry, possible countermeasures, and the overall implications of software interfaces for modern on-chip power management systems.

ACM Reference Format:

Nikhil Chawla, Chen Liu, Abhishek Chakraborty, Igor Chervatyuk, Ke Sun, Thaís Moreira Hamasaki, Henrique Kawakami. 2024. Uncovering Software-Based Power Side-Channel Attacks on Apple M1/M2 Systems. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3656531>

1 Introduction

It is well-known that CMOS circuits, when processing data, generate data-dependent power consumption. This behavior has been misused to perform power analysis attacks, which extracts sensitive data, such as secret keys or passwords, from the target system. Traditionally, performing power side-channel attacks requires the attacker to have physical access to the system for power consumption measurement. Software-based power side-channel attacks represent a new class of power side-channel attacks that can be performed by

a software attacker leveraging on-chip power meter or power management capabilities provided by the hardware, without requiring physical access to the system. One example of such attacks is the *PLATYPUS* [1], which leverages the Running Average Power Limit (RAPL) energy counters available on x86 CPUs to extract sensitive information. Another example is frequency throttling attack [2, 3], which further converts the power side-channel to frequency/timing side-channel, allows information leakage even without exposing power meter interfaces.

While most of these researches are targeting x86 CPUs, researches on applicability of such attacks targeting other architectures is limited. In this paper, for the first time, we perform a comprehensive analysis of software-based power side-channel attack targeting ARM-based Apple M1/M2 systems. Specifically, we observed that the System Management Controller (SMC) on Apple silicon exposes power meter capabilities to software, and a set of such metrics are even accessible to user-space application in macOS. With experiments, we identified the power meters that are correlated with the data processed by the CPU. Furthermore, we show that the data-dependency can be exploited by user-space software that perform power side-channel attacks to extract secrets (e.g., AES [4] keys) from a privileged software (e.g., kernel drivers). Besides that, we have also studied on an alternative power reporting interface and the feasibility of performing throttling side-channel attacks on Apple silicon. Our key contributions in the paper are:

- To the best of our knowledge, this is the first work that presents comprehensive analysis on feasibility of software-based power side-channel attacks on ARM-based Apple silicon, including both power-meter-based attack [1] and frequency throttling attack [2, 3].
- We discover that specific power meters provided by the SMCs on the Apple silicon are data-dependent and prove that with experiments. This is the first work that shows the existence of such side-channel. Moreover, we show that those power meters, which are exposed to a user mode application, allow an unprivileged attacker to infer secret (e.g., AES key) from kernel mode. Our PoC demonstrates complete leakage of 6 key bytes and nearly-complete leakage of the other 6 key bytes for a 16-byte AES-128 key.
- We discuss the impact of software-based power side-channels on the industry as a whole and possible countermeasure techniques to mitigate the risks.

The remainder of this paper starts by presenting background and related work in Section 2. In Section 3, we present the details of our research on finding the data-dependency of the SMC power meters reporting, as well as the exploitation of the power side-channel to extract secret. We further discuss the experiments on throttling

side-channel feasibility in Section 4 and discuss the possible countermeasure techniques in Section 5. Section 6 concludes our paper.

Responsible Disclosure: We responsibly disclosed our findings to Apple Inc. on November 29th, 2022. Following up on Apple’s request, we provided two different Proof-of-Concept (PoC) versions on January 2023 and March 2023, respectively. Apple acknowledged the findings, validated the PoC on June 27, 2023 and did not propose to take further actions to mitigate the reported issue.

2 Background & Related Work

2.1 Power meter reporting on Apple platforms

The System Management Controller (SMC) is a co-processor responsible for power and thermal management available in multiple Apple platforms since the legacy x86-based Apple systems[5]. The SMC exposes various sensor data including temperature, voltage and power meters, battery status, fan status, and other power-related functions on the system. The sensor data is accessible to user mode software as a key-value pair, where the key is a 4-character alphanumeric string[6] and the value can be retrieved by calling the `IOConnectCallStructMethod` function in the `IOKit`[7] library built in macOS.

2.2 Software-based power side-channel attacks

In a traditional physical power side-channel attacks, the attacker measures the data-dependent power consumption of cryptographic implementations processing sensitive data and perform statistic-based analysis to recover the secret [8]. Correlation Power Analysis (CPA) is one of the traditional power side-channel analysis techniques that computes correlation coefficient between the actual secret dependent power traces and hypothetical power model to recover to secret key.

Unlike the traditional physical side-channel attacks, software-based power side-channel attacks do not require physical access and can be performed remotely by exploiting information provided by underlying hardware through software accessible interfaces. The *PLATYPUS* attack [1] has shown that the Running Average Power Limit (RAPL) energy reporting interfaces, which are available on Intel and AMD processors, expose data-dependent power consumption information that can be used to perform power side-channel attack by a software attacker. Furthermore, recent works [2, 3] have discovered that CPUs, when hit certain reactive limits (e.g. power or current limits), will throttle to run at lower frequencies, which are correlated with energy consumption of the chip and hence correlated with data processed by the CPU. *Collide+Power* [9] further shows that data-dependent power consumption information from the memory hierarchy can be exposed through those channels. Taneja *et al.* [10] discusses thermal-limit-induced frequency throttling side-channel on various integrated and discrete GPUs, and also mentions software-based power side-channels on ARM-based CPUs, including Apple M1/M2 systems. Compared to [10], this work conducts a more comprehensive study on the software-based power side channel on Apple M1/M2 platforms, and more importantly, finds new software-accessible interfaces that exhibits more apparent data-dependent side-channel leakage.

Table 1: Specifications of the tested devices

Devices	P-cores		E-cores		OS version
	count	max freq.	count	max freq.	
Mac Mini M1	4	3.2 GHz	4	2.4GHz	macOS 12.5
Mac Air M2	4	3.5 GHz	4	2.06GHz	macOS 13.0

Table 2: Workload-dependent SMC keys

SMC keys	Mac Mini M1			Mac Air M2		
	PDTR, PMVR,	PHPC, PPMR	PHPS, PSTR	PDTR, PMVC,	PHPC, PSTR	PHPS

3 SMC power meter side-channel attacks

In this section, we provide detailed descriptions of the procedures used to identify data-dependent SMC key values and evaluate the feasibility of performing software-based side-channel attacking using the keys. We used an Apple Mac Mini M1 and an Apple MacBook Air M2 for all the experiments described in this paper, and their specifications are summarized at Table 1.

3.1 Threat model

In this work, we assume the attacker is a user mode software and the victim is a kernel mode driver holding a secret. The victim may provide services to user mode software. The attacker might use these services to call/invoke kernel routines that operate on the secret data. The attacker, as a user mode application, has no direct access to the secret which belongs to the kernel mode driver. We assume the kernel mode driver implementation follows secure coding guidelines so it is not vulnerable to traditional timing side-channel attacks.

3.2 Identifying power-correlated SMC keys

Since official SMC key definitions for Apple M1/M2 systems are not publicly available, the initial phase of the research involved identifying the specific SMC keys that correlate with power consumption among the extensive collection of available keys. Based on the established naming convention adopted in x86-based Mac systems, it has been observed that SMC keys associated to power-related functionalities commonly start with an initial capital letter “P” [11]. By leveraging this information, we were able to significantly narrow down the pool of candidate keys to approximately 30.

In the next step, we used an open source tool *smc-fuzzer* [12] to enumerate values of the candidate keys when the system is idle and busy, respectively. For all the SMC keys on the Apple M2 system that start with the letter “P”, we collected their corresponding values when the system is in an idle state, as well as when a stressor *stress-ng* [13] workload, which performs matrix operations on all available cores, is running. These two scenarios are chosen because they are expected to show large power difference. By conducting a side-by-side comparison between the results under both scenarios, we identified several SMC keys that exhibit large variations in their values. Those keys are considered correlated with power and are selected as the candidates for the next step analysis. The same experiment was also replicated on Apple M1. Table 2 summarizes the list of SMC keys identified on both systems, respectively.

Table 3: TVLA analysis on Apple M2 system for selected SMC keys between different plaintexts for the user space AES workload

SMC key	PHPC			PDTR			PHPS			PMVC			PSTR		
Plaintext	All 0s	All 1s	Random	All 0s	All 1s	Random	All 0s	All 1s	Random	All 0s	All 1s	Random	All 0s	All 1s	Random
All 0s'	-0.18	20.94	11.49	8.73	29.58	25.05	0.87	2.14	2.02	9.49	32.16	27.33	21.30	12.96	27.41
All 1s'	-21.09	0.09	-8.87	-20.42	0.02	-4.49	-1.97	-0.69	-0.84	-22.45	0.09	-5.15	9.13	0.37	15.16
Random'	-11.60	9.28	0.43	-15.28	5.01	0.55	-0.53	0.74	0.61	-17.54	4.92	-0.23	5.99	-15.03	-0.24

3.3 Identifying data-dependent SMC keys

Our next step is to identify the data-dependent SMC keys from the previous list of power-correlated keys. To accomplish this, we implemented a Proof-of-Concept (PoC) that executes a workload which accepts and processes distinct input data for a fixed number of iterations. The workload is constant-cycle to avoid impact from timing side-channel. Values of all the selected SMC keys are measured and logged in a *trace* after the workload finishes. The process is repeated to collect multiple *traces* containing the SMC key values.

In order to make the potential data-dependency more observable, we opted to replicate the workload and execute it simultaneously on three P-cores. The three copies execute the same workload using the same input data at any time, therefore the data-dependent power consumption is amplified. We did not run additional copy on the fourth performance cores (P-core) or the efficiency cores (E-cores). We applied the Test Vector Leakage Assessment (TVLA) [14] to validate if any of the pre-selected SMC key value traces show data-dependency. TVLA utilizes Welch's t-test to assess side-channel leakage of cryptographic implementations. A Welch's t-test compares two datasets, A and B, by computing a statistic score, the *t-score*. A $|t\text{-score}| > 4.5$ indicates that the two datasets are statistically distinguishable with 99.999% confidence. For this experiment, we selected the implementation of AES-128 encryption from *AES-Intrinsics* [15] as the testing workload, which utilizes the ARMv8 equivalent to the cryptographic extension AES instructions (e.g., *AESE* and *AESMC*). We observed that the SMC key values are updated approximately every one second. Therefore, in the workload we repeated the AES encryption of the same plaintext so it takes slightly longer than a second. We collected 10k SMC key value traces corresponding to the encryption of each of the three chosen plaintexts - (*All_0s*, *All_1s*, and *Random*) - with a fixed key. We then applied TVLA between every possible pair of the chosen plaintexts. Table 3 shows the calculated *t-score* on the Apple M2 system, where different colors mean:

- **True positive:** two traces with different plaintexts are distinguishable ($|t\text{-score}| \geq 4.5$).
- **True negative:** two traces with the same plaintext are non-distinguishable ($|t\text{-score}| < 4.5$).
- **False positive:** two traces with the same plaintext are distinguishable ($|t\text{-score}| \geq 4.5$).
- **False negative:** two traces with different plaintexts are non-distinguishable ($|t\text{-score}| < 4.5$).

If a SMC key is data-dependent, we expect it shows more true positive/negative and less false positive/negative. Out of the selected SMC keys, one key, namely *PHPC*, stands out by demonstrating true positive and true negative results consistently. Remarkably, *PHPC* exhibits no instances of false positive or false negative correlations. This finding strongly suggests that *PHPC* has the most robust and reliable correlation with the data. In the case of *PDTR*, *PMVC*, and *PSTR*, these SMC keys exhibit a combination of true positive and

true negative results for a majority of the pairs. However, they also display several instances of false positive or false negative. This indicates a somewhat weaker data-correlation compared to *PHPC*. Conversely, the SMC key *PHPS* primarily generates false negative correlations for most of the pairs and yields no true positive correlations, suggesting a limited or no data-correlation. As a result of our analysis, we have confirmed the data-dependency for all the selected SMC keys except for *PHPS* on Apple M2 system. Additionally, we conducted TVLA on the collected traces of the *PHPC* key values collected on the Apple M1 platform, affirming a similar data-dependency pattern for the *PHPC* key on that system as well.

3.4 AES encryption key extraction

Following the confirmation of data-dependency in the SMC key values, we then assessed the feasibility of extracting secrets from a victim program. In a manner similar to the previous experiment, we used a victim program performing AES-128 encryption repeatedly, which carries out encryption operations using a secret key that remains inaccessible to the attacker. We still assume the attacker can perform *known-plaintext attack*, in which he/she controls the plaintext. In contrast to providing fixed plaintexts like the TVLA analysis, the attacker now provides random plaintext inputs into the victim program. During this process, the attacker records the plaintext, the generated ciphertext, and the corresponding SMC key values right after the encryption operation. As a starting point, the AES-128 encryption takes place in user mode of this PoC for the purpose to minimize noise. We will discuss extracting kernel secret later.

In this experiment, we collected 1 million SMC key value traces on the M2 system and 350 thousand traces on the M1 system, respectively. We then performed Correlation Power Analysis (CPA) to analyze the collected traces. CPA involves computing correlations between the power side-channel traces and a hypothetical power model derived from the Hamming Weight (HW) or Hamming Distance (HD) of intermediate states. In our CPA approach, the power traces used for analysis correspond to the SMC key value traces we collected. The hypothetical power models employed are similar to those used in traditional CPA. These models incorporate the HW or HD of intermediate states, which are:

- **Rd0-HW:** HW after the first *AddRoundKey* operation to recover initial round key
- **Rd10-HW:** HW before the last round *SubBytes* operation to recover round #10 key
- **Rd10-HD:** HD between last round input and ciphertext to recover round #10 key.

For each targeted key byte, the hypothetical power model is constructed using HW/HD of intermediate round state for all possible key guesses. We calculated the correlation coefficient between the SMC key value trace and the hypothetical power model, then ranked all key guesses in descending order based on correlation coefficient.

Table 4: Rank of each AES key byte applying CPA on collected SMC key value traces with Round 0 HW power model on Macbook Air M2 and M1 (PHPC only)

#key byte	PHPC	PDTR	PMVC	PSTR	PHPC (M1)
0	7	1	3	211	9
1	7	7	3	22	19
2	1	5	3	188	4
3	11	11	12	189	12
4	5	1	1	151	1
5	4	15	12	223	31
6	4	6	14	113	16
7	13	8	12	39	5
8	1	15	17	201	9
9	37	16	22	101	18
10	1	5	11	214	7
11	1	2	2	117	2
12	1	2	1	146	1
13	4	12	13	184	36
14	1	9	8	18	25
15	26	24	14	137	50
GE	31.0	41.6	42.8	109.3	40.9

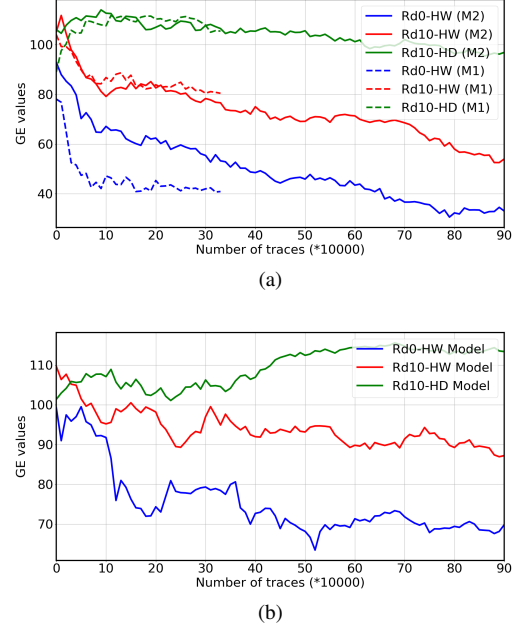
The outcome of the CPA test is the rank of the correct key byte among all key guesses, with a value of 1 indicating successful recovery of the secret key byte. The average rank of all correct key bytes is measured by *Guessing Entropy* (GE). Lower GE indicates lower ranks in general across all key bytes and $GE = 0$ indicates recovery of all key bytes.

Table 4 presents the rank of each secret key byte and the *Guessing Entropy* (GE) using the *Rd0-HW* power model on M1/M2 systems. Key bytes with a rank of 1, indicating successful recovery, are highlighted in red. Those with ranks below 10, suggesting near recovery, are in yellow. On the Macbook Air M2, CPA analysis of *PHPC* traces successfully recovers 6 key bytes and nearly recovers another 6. For *PDTR* and *PMVC* traces, CPA yields low ranks for most key bytes (10/16 for *PDTR*, 7/16 for *PMVC*). However, CPA analysis of *PSTR* traces fails to recover any key bytes. On the Apple M1 mini, CPA testing of the *PHPC* key successfully recovers 2 bytes and nearly recovers 6 others, consistent with prior TVLA findings.

Figure 1(a) illustrates how the GE varies with the number of *PHPC* key value traces and different power models for both M1 and M2 systems. The GE trend for the M1 system is shorter due to a smaller dataset of *PHPC* traces (350K traces). The trends suggest that accumulating more traces improves the likelihood of recovering all key bytes. Different power models show varying convergence patterns. The *Rd0-HW* model converges quickest, highlighting its superior key recovery capability. The *Rd10-HW* model converges as well, but more slowly. In contrast, the *Rd10-HD* model shows little convergence, indicating it is less effective for key recovery. These findings affirm the *Rd0-HW* model’s effectiveness and dependability for extracting information from SMC key value traces on Apple’s M1/M2 platforms.

3.5 Targeting a kernel module

To assess the attack’s viability across different security levels, we implemented a kernel module on the M2 system to act as the target program, with the attacker being a user space application that has read access to the SMC key values via the *IOKit* framework’s interface. This victim kernel module functions as an encryption service, taking plaintext from a user application, performing encryption

**Figure 1: GE trend against collected *PHPC* SMC key value traces for CPA targeting (a) user space AES encryption on Apple M1 Mini and M2 Air systems, and (b) AES kernel module on the Macbook Air M2 system**

repeatedly for a set number of cycles, and then storing the resulting ciphertext in a buffer accessible to the user application. In our setup, a single P-core thread triggers the device driver to encrypt the plaintext.

We applied TVLA to the SMC keys collected to assess data dependency, as outlined in section 3.3. We recorded SMC key traces while encrypting different plaintext inputs using the device driver. The *t-scores* for these traces during fixed-key encryption on the MacBook Air M2 are displayed in Table 5, with color-coding for TVLA results following the standards set in section 3.2. The data dependency patterns observed were consistent with those from a user space victim, previously discussed. The *PHPC* key traces demonstrated the most significant data dependency, with *PDTR*, *PMVC*, and *PSTR* also showing dependency, and *PHPS* traces revealing the least correlation.

We further conducted a CPA test on the SMC keys whose values exhibit data correlation in the TVLA test. The objective is to recover the AES encryption key from the device driver. We collected one million traces for each of the SMC keys and then apply CPA with the hypothetical power models considered in section 3.3 to obtain the GE metrics. Fig. 1(b) illustrates the GE trend with different power models against the one million traces collected for *PHPC* SMC key. We observed a converging trend indicating a reduction in the rank of correct key bytes with increasing number of collected traces. Among the power models tested, the *Rd0-HW* power model demonstrates the strongest correlation indicated by the fastest convergence of the GE metric, while the *Rd10-HD* power model do not exhibit any convergence. Additionally, we observed that the GE metric in Fig.

Table 5: TVLA analysis on selected SMC key traces corresponding to encryption of different plaintext inputs to AES kernel module on the Macbook Air M2 system

SMC key	PHPC			PDTR			PHPS			PMVC			PSTR		
Plaintext	All 0s	All 1s	Random	All 0s	All 1s	Random	All 0s	All 1s	Random	All 0s	All 1s	Random	All 0s	All 1s	Random
All 0s'	2.78	19.28	9.41	13.84	41.52	43.01	2.72	3.60	6.51	15.13	45.38	47.10	40.66	18.45	37.50
All 1s'	-17.91	-0.76	-11.12	-30.27	-2.16	-0.26	-3.99	-3.12	-0.11	-32.93	-2.44	-0.48	-18.45	1.66	20.01
Random'	-6.77	10.14	-0.04	-30.84	-0.73	-0.99	-3.85	0.11	0.03	-33.30	-0.56	-1.03	0.73	-20.70	-2.16

Table 6: TVLA analysis on the 'PCPU' channel traces (from IOReport) and execution time traces (during lowpowermode throttling) on Macbook Air M2 system

Plaintext	PCPU (IOReport)			Time (during throttling)		
	All 0s	All 1s	Random	All 0s	All 1s	Random
All 0s'	-0.34	-1.41	-2.68	-0.34	-1.07	-0.29
All 1s'	2.08	0.28	-1.95	0.84	0.10	0.90
Random'	-2.68	1.95	-1.09	-0.41	-1.13	-0.36

1(b) converges approximately two times slower compared to the user mode AES application (Fig. 1(b)). This can be attributed to the decreased *Signal-to-Noise Ratio* (SNR) of the collected traces due to system call invocations, as well as the lower number of victim threads in the kernel module test. The experimental findings from the CPA test indicate that an unprivileged attacker can compromise the confidentiality of assets protected by the kernel using SMC key as a side-channel.

3.6 IOReport Energy Reporting

We have also examined alternative power reporting interfaces beside the SMC keys. We leveraged an open-source tool `socpowerbud` [16] which utilizes `IOReport` functions imported from `Foundation.h` header file to dump system level statistics for user space software to read. These statistics are further categorized into multiple group, where each group has multiple channels. Among all the groups, we find that "PCPU" channel within "Energy Model" group reports energy consumption of the P-cores, which is potentially another exploitable power side-channel¹. We assess the side-channel leakage from the collected "PCPU" channel readings by applying TVLA to check if it exhibits data-dependency. The result (shown at Table 6 first column), however, indicates no data correlation as there are all false negatives between different plaintext pairs. We attribute the observation to two reasons. first, "PCPU" channel reports energy with resolution of mJ, so that the averaged power has a coarser-grained resolution (mW) compared to that derived from SMC keys (uW); more importantly, we suspect "Energy Model" group reports estimated energy model calculated based on core utilization instead of actual sensor reading, therefore doesn't reveal data-dependency.

4 Throttling side-channel analysis on M2

We also explored the viability of conducting frequency throttling side-channel attacks on the Apple M2 system, another variant of software-based power side-channel attacks previously identified in x86 platforms. This method requires the system to hit a reactive

power limit (e.g., power limit) while the victim program is operational. To trigger this, we ran multiple instances of the victim program across various cores, augmenting with stressor code on the other cores where necessary to exceed the system's power threshold. However, with such settings we observed the system's thermal limit was consistently reached before any power-based throttling. While reaching thermal limit also triggers frequency throttling and previous work [10] shows it reveals certain level of side-channel information, we found that its SNR is too low to mount any realistic attack.

Finding Reactive Power Limits: The observations outlined above prompted us to explore strategies for reaching the power limit without triggering the thermal limit. One potential method involves lowering the default power limit. However, we encountered a lack of publicly accessible documentation regarding power management and specific reactive limits for the M2 system. Our investigation of modifiable System Management Controller (SMC) keys, through visual inspection and the use of the `smc-fuzzer` tool [12], did not reveal any keys related to reactive limit configurations.

Further analysis using `pmset`, a utility provided by Apple's macOS for adjusting power management settings, led us to discover a tunable binary setting named `lowpowermode`. Activating `lowpowermode` by setting it to 1, we conducted tests to observe the effects on CPU power and core frequencies under varying workload conditions. Starting with a single AES-128 encryption thread on one P-core, we incrementally increased the load by adding identical threads across more cores. We noted that the P-cores maintained a consistent frequency of 1.968 GHz when the total CPU power consumption remained under 4W. Upon attempting to increase the workload further, we observed a power cap at 4W, coinciding with the onset of frequency throttling in the P-cores. Given that the CPU temperature remained low during this throttling, we ruled out thermal factors as the cause and concluded that a power limit of 4W is enforced in `lowpowermode`, with P-core throttling occurring upon reaching this threshold. Interestingly, the efficiency cores (E-cores) did not exhibit throttling and continued to operate at a stable frequency of 2.424 GHz.

Triggering Frequency Throttling: Upon identifying a method to induce power limit-based throttling, we aimed to harness this for a potential side-channel attack. Two prerequisites were established: 1) secret-dependent computations must be confined to the P-cores, as the E-cores do not experience throttling, and 2) the CPU's power consumption must surpass the 4W threshold set by the `lowpowermode`. Drawing from prior experiments, we chose an AES-128 encryption routine from AES-Intrinsics [15] as our target. In `lowpowermode`, running the AES-128 workload on all four P-cores resulted in a power draw of only 2.8W, which is insufficient to induce throttling. Consequently, we introduced additional stressors on the E-cores to elevate the system's power usage. The challenge

¹Please note that "PCPU" is not a SMC key, although it aligns with SMC key's naming convention.

was to confine the AES threads to the P-cores while running stressors on the E-cores. We addressed this by adjusting the scheduler policy to round-robin (`SCHED_RR`) and setting the AES threads to the highest priority, guiding the operating system's scheduler to favor the P-cores for these threads. Our stressor code executed floating-point multiplication (`fmul`) instructions between two constant operands, creating a steady and secret-independent power load that increased the system's total power consumption without introducing power fluctuations. Our experiments showed that by operating four AES threads on the P-cores alongside a `fmul` stressor on the E-cores, we reached the 4W power ceiling, triggering throttling on the P-cores.

Experimental Results: Adopting the setup outlined earlier, we replaced SMC key measurements with execution time the AES threads, as detailed in Section 3.2. We conducted TVLA analysis on the timing traces with results presented in the second column of Table 6. The absence of true positives suggests that the timing (and throttled frequency) is not data-dependent. Notably, the "PHPS" SMC key value also peaked at 4W during throttling. Given the lack of data-dependency in "PHPS" values (Table 3), we infer that `lowpowermode`'s frequency throttling may rely on "PHPS" rather than actual power use, explaining the lack of data correlation.

5 Countermeasures & Discussions

To address the vulnerabilities highlighted by the PLATYPUS attack [1], both Intel and AMD have taken measures to remove the user space software's access to the RAPL (Running Average Power Limit) energy counters from the Linux kernel drivers [17, 18]. Additionally, Intel has introduced a RAPL filtering mechanism, which blends random energy noise with the energy reporting and adjusts the update interval, aiming to lower the SNR of power consumption measurements and hinder the extraction of sensitive data [19].

In light of our research, we propose that analogous countermeasures could be effective for the SMC key power side-channel vulnerabilities we've identified. Restricting user space access to power-related SMC keys would prevent user space attackers from exploiting these values. Incorporating noise into power consumption readings could further degrade the SNR, diminishing the threat posed by these side-channel attacks. As of our publication, there is no indication that Apple has implemented specific mitigation against the vulnerabilities we've discussed. It is imperative for system vendors, including Apple, to consider the security implications of power side-channel attacks and to establish robust defenses to secure sensitive information.

Our study contributes to a growing amount of work that demonstrates the risk of software-based power side-channel attacks across different CPU architectures. These findings highlight an industry-wide issue, stressing the importance for all vendors to acknowledge the threat and to actively seek out architectural solutions and preventative measures to counteract emerging software-based power side-channel attacks.

6 Conclusion

This paper has explored the vulnerabilities arising from software-based power side-channel attacks and their implications across CPU architectures. Our experiments and analysis have revealed the data dependency of the power consumption reported by the selected

System Management Controller (SMC) key values on ARM-based Apple Silicon M1/M2 systems. Furthermore, we demonstrated that a user space attacker can exploit the unprivileged access to the SMC key values to extract secrets from the kernel device driver. Our research underscores the need for industry-wide awareness and proactive exploration of architectural solutions to prevent software-based power side-channel attacks.

References

1. Lipp, M. *et al.* *PLATYPUS: Software-based Power Side-Channel Attacks on x86 in 2021 IEEE Symposium on Security and Privacy (SP)* (2021), 355–371.
2. Wang, Y. *et al.* *Hertzbleed: Turning Power Side-Channel Attacks Into Remote Timing Attacks on x86 in 31st USENIX Security Symposium (USENIX Security 22)* (USENIX Association, Boston, MA, Aug. 2022), 679–697.
3. Liu, C. *et al.* *Frequency Throttling Side-Channel Attack in Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (Association for Computing Machinery, Los Angeles, CA, USA, 2022), 1977–1991.
4. National Institute of Standards and Technology. *ADVANCED ENCRYPTION STANDARD (AES)* tech. rep. (U.S. Department of Commerce, Washington, D.C., 2001).
5. Barisani, A. *Practical Exploitation of Embedded Systems* (Accessed June 8, 2023). https://github.com/abarisani/abarisani.github.io/blob/master/research/smc/practical_exploitation_of_embedded_systems.pdf.
6. Ionescu, A. *Ninjas and Harry Potter - "Spell" unking in Apple SMC Land* (Accessed June 8, 2023). https://nosuchcon.org/talks/2013/D1_02_Alex_Ninjas_and_Harry_Potter.pdf.
7. Apple Inc. *Apple Developer Documentation - IOKit* (Accessed June 8, 2023). <https://developer.apple.com/documentation/iokit/1514274-ioconnectcallstructmethod>.
8. Mangard, S. *et al.* *Power analysis attacks: Revealing the secrets of smart cards* (Springer Science & Business Media, 2008).
9. Kogler, A. *et al.* *Collide+Power: Leaking Inaccessible Data with Software-based Power Side Channels in USENIX Security* (2023).
10. Taneja, H. *et al.* *Hot Pixels: Frequency, Power, and Temperature Attacks on GPUs and ARM SoCs 2023*. arXiv: 2305.12784 [cs.CR].
11. *VirtualSMC* (Accessed June 2, 2023). <https://github.com/acidanthera/VirtualSMC/tree/master>.
12. Teddy Reed, b. *SMC Fuzzer* <https://github.com/theopolis/smc-fuzzer>. (Accessed May 26, 2023).
13. King, C. I. *et al.* *stress-ng* (Accessed Nov 18, 2023). <https://github.com/ColinIanKing/stress-ng>.
14. Goodwill, G. *et al.* *A testing methodology for side-channel resistance validation in NIST non-invasive attack testing workshop 7* (2011).
15. Jeffrey Walton, R. Z. *AES-Intrinsics* <https://github.com/noloader/AES-Intrinsics/tree/master>. (Accessed May 26, 2023).
16. Teddy Reed, b. *socpowerbud* <https://github.com/dehydratedpotato/socpowerbud>. (Accessed May 26, 2023).
17. Intel Inc. *2020.2 IPU - Intel RAPL Interface Advisory* <https://www.intel.com/content/www/us/en/security-center/advisory/intel-sa-00389.html>. (Accessed June 9, 2023).
18. MITRE. *CVE-2020-12912* <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-12912>. (Accessed June 9, 2023).
19. Intel Inc. *Running Average Power Limit Energy Reporting / CVE-2020-8694, CVE-2020-8695 / INTEL-SA-00389* <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/running-average-power-limit-energy-reporting.html>. (Accessed June 9, 2023).

Q-Pilot: Field Programmable Qubit Array Compilation with Flying Ancillas

Hanrui Wang^{1*}, Daniel Bochen Tan^{2*}, Pengyu Liu³, Yilian Liu⁴, Jiaqi Gu⁵, Jason Cong², Song Han¹

¹MIT, ²University of California, Los Angeles, ³Carnegie Mellon University, ⁴Cornell University, ⁵Arizona State University, *Equal Contributions

ABSTRACT

Neutral atom arrays, particularly the reconfigurable field programmable qubit arrays (FPQA) with atom movement, show strong promise for quantum computing. FPQA has a dynamic qubit connectivity, facilitating cost-effective execution of long-range gates, but it also poses new challenges in the compilation. Inspired by the FPGA compilation strategy, we develop a router, Q-Pilot, that leverages flying ancillas to implement 2-Q gates between data qubits mapped to fixed atoms. Equipped with domain-specific routing techniques, Q-Pilot achieves 1.4 $\times$, 27.7 $\times$, and 6.7 $\times$ reductions in circuit depth for 100-qubit random, quantum simulation, and QAOA circuits, respectively, compared to alternative fixed atom array architectures.

1 INTRODUCTION

Quantum computing (QC) hardware has seen rapid scaling, with superconducting systems offering up to 433 qubits [1–4], and neutral atom arrays reaching 1000+ qubits [5, 25]. Utilizing these machines requires mapping qubits in a quantum program/circuit to physical qubits on the QPU, typically constrained by limited connectivity given by a coupling graph. For example, Fig. 1a illustrates a simple QPU with four physical qubits connected in a ring. 2-Q entangling gates, crucial for quantum programs, are restricted to adjacent physical qubits (e.g., (p_0, p_1)). Consider a quantum program with gates $CZ(q_0, q_1)$, $CZ(q_1, q_2)$, and $CZ(q_2, q_0)$. In Fig. 1b, the initial qubit mapping is $q_i \mapsto p_i$ for $i = 0, 1, 2$. While this mapping supports the first two gates, $CZ(q_2, q_0)$ involves non-adjacent p_2 and p_0 . Here, a SWAP gate is inserted to *route* qubits, transforming the mapping. However, SWAP is costly: it can increase circuit depth, leading to more decoherence noise, and typically requires three 2-Q entangling gates, accumulating gate errors. Given the current QPUs' relatively high noise levels, as quantum circuits grow, it becomes crucial that compilers minimize the overheads incurred by mapping and routing to optimize performance [7, 8, 14, 18, 22–24, 26, 29–31, 34–37].

A recent breakthrough enables atom movement during quantum circuit execution [10], profoundly impacting compilation by introducing dynamic coupling graphs for QPUs, as opposed to static configurations (Fig. 1). In this work, we focus on a field programmable qubit array (FPQA) architecture that incorporates this technology. FPQA features two atom types (Fig. 2): SLM atoms (blue) are *fixed* atoms in traps generated by a spatial light modulator (SLM); AOD atoms (yellow) are *movable* atoms in traps generated by a 2D acousto-optic deflector (AOD). The 2D AOD, a product of two 1D AODs, allows us to specify X coordinates for columns (yellow dashes) and Y coordinates for rows. Consequently, AOD qubits move *by entire rows and columns*. To avoid non-deterministic behavior from trap overlap, we *prohibit AOD rows/columns from moving over others*. Physically, atom movement is a high-fidelity operation primarily constrained by coherence time: with only 0.1% coherence time, an atom can traverse

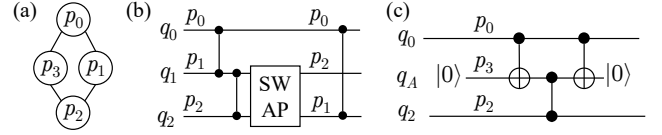
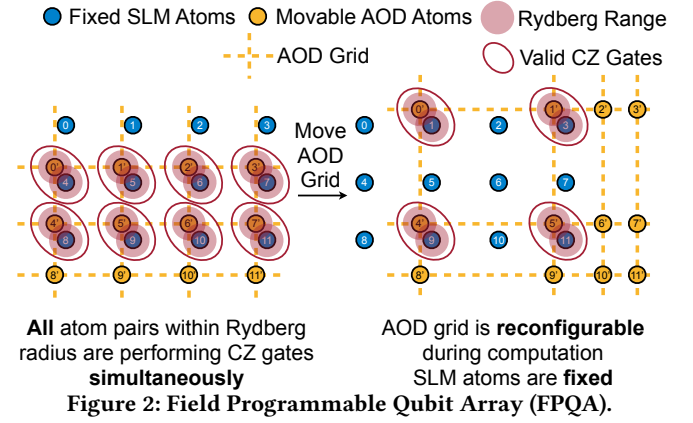


Figure 1: (a) The coupling graph of a QPU. (b) Qubit mapping and routing. The initial mapping is annotated at the beginning of each wire/qubit. A SWAP gate changes the mapping. (c) Using an ancilla and two more CNOTs to implement $CZ(q_0, q_2)$.



a region for $\sim 2,000$ qubits [10]. These movements are explicitly applied for 2-Q gates, which are induced by a *global* Rydberg laser activating all atoms. If two qubits are within the *Rydberg radius* r_b , they become ‘coupled,’ enabling the application of a CZ gate by the Rydberg laser. Moving atoms between circuit stages couples different qubit pairs, resulting in a dynamic coupling graph. To avoid unintended 2-Q gates, other atoms must be *sufficiently separated* ($> 2.5r_b$). The global Rydberg laser requires less control and calibration, enhancing the *scalability* of FPQA compared to prior works [16, 28] where the laser *individually* address qubits for 2-Q gates.

We introduce Q-Pilot, the first scalable FPQA router drawing inspiration from FPGA placement and routing. Our approach, termed “routing with flying ancillas,” involves qubit mapping akin to cell placement and the use of movable ancilla qubits to bridge fixed atoms, similar to FPGA routing. The advantages of flying ancillas include 1) high-parallelism circuit execution, 2) scalable compilation, and 3) no atom transfer required during computation. To boost parallelism and thus reduce circuit depth, we implement a high-parallelism generic router, dynamically arranging AODs and scheduling 2-Q gates. The router heuristically schedules as many parallel executable gates as possible in one laser stage up to AOD movement constraints. We also devise *application-specific* strategies: for each Pauli string in quantum simulation, we create multiple ancillas for a “root” qubit and employ graph algorithm to find the longest chain in the SLM array to perform the gates; for QAOA, we create ancilla per qubit instead of per gate, and leverage the commutation of gates to maximize parallel execution and reduce depth. Extensive experiments demonstrate that our FPQA compilation framework outperforms the best baseline, achieving 1.4 $\times$, 27.7 $\times$, and 6.7 $\times$ smaller circuit depth for 100-qubit random, quantum simulation, and QAOA circuits.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0601-1/24/06.

<https://doi.org/10.1145/3649329.3658470>

2 RELATED WORKS

Previously, research focused on fixed atom arrays with static coupling. Ref. [9] introduced the first compiler framework for this architecture, extending existing techniques for superconducting devices and addressing unique constraints, including long-range interaction restriction zones and sporadic atom loss. Ref. [20] further considers gate durations in compilation. Geyser (Ref. [27]) leverages native 3-Q operations by blocking 3-Q sub-circuits and re-synthesizing them.

Ref. [11] first considered atom movement, exploring a hypothetical architecture with ‘1D displacement’ capability, more restrictive than FPQA. Ref. [32] presented the first optimal 2D FPQA compiler, formulating constraints and using an SMT solver for qubit mapping and routing. However, the solver-based method’s scalability is limited by the exponential SMT solving. Their updated work [33] improves the scalability at some expense of optimality and reached the scale of about 100-qubit circuits within a day. Additionally, it is worth noting that they employ *atom transfer* operations, moving an AOD atom to an empty SLM trap when in proximity and vice versa. While atom transfer is already utilized in experiments [12], frequent transfers ‘heat up’ the atoms, potentially resulting in atom loss errors.

3 FLYING ANCILLAS

3.1 Motivating Example of Routing CZ

Revisiting the issue of the last gate in Fig. 1, (c) introduces an alternative using ancilla qubit q_A at p_3 instead of the SWAP: q_A is initialized to $|0\rangle$, the three-qubit initial state (in order $q_0 q_A q_2$) can be written as $a|000\rangle + b|001\rangle + c|100\rangle + d|101\rangle$. After the first CNOT, it becomes $a|000\rangle + b|001\rangle + c|110\rangle + d|111\rangle$. After the CZ, it becomes $a|000\rangle + b|001\rangle + c|110\rangle - d|111\rangle$. After the second CNOT, it is $a|000\rangle + b|001\rangle + c|100\rangle - d|101\rangle$, which is the same as the case where $CZ(q_0, q_2)$ acts on the initial state. In this process, q_A acts as a ‘fan-out’ of q_0 . However, note that it is only on the Z basis. Hence, this method’s effectiveness hinges on the targeted 2-Q gate, specifically CZ in our case (and ZZ later on), but it is not universally applicable. Thus, we decompose other 2-Q gates using CZ or ZZ beforehand. Some previous works [15, 17] leveraged these fan-outs to reduce cost in circuit synthesis, but we apply them in routing because uniquely in FPQA, the fan-out qubits can move physically. If we rely on SWAPs for the routing, the depth increases by 3 because we need 3 CNOT for 1 SWAP, yet the new approach only increases depth by 2.

3.2 General Theory of Routing CZs with Ancillas

We prove a general result independent of the coupling graph. Given an arbitrary n -qubit state $\Psi = C_0|0\rangle + C_1|1\rangle + \dots + C_{2^n-1}|2^n-1\rangle$, and a set of qubit pairs C , applying $CZ_{j,j'} \forall (j, j') \in C$ yields

$$\Psi' = \left(\prod_{(j,j') \in C} CZ_{j,j'} \right) \Psi = \sum_{x=0}^{2^n-1} C_x \prod_{(j,j') \in C} (-1)^{x_j x_{j'}} |x\rangle, \quad (1)$$

where x_j is the j -th bit of x . We consider an alternative procedure as illustrated in Fig. 3 where we 1) apply transversal CNOTs from the n qubits to n fresh ancillas yielding Φ_1 , 2) apply one of the four possibilities (2 choices of whether to $+n$ for 2 indices) $CZ_{j(+n),j'(+n)} \forall (j, j') \in C$ yielding Φ_2 , and 3) apply transversal CNOTs again yielding Φ_3 . We prove that $\Phi_3 = \Psi' \otimes |0^n\rangle$, so our procedure is equivalent to applying the original CZs in Eq. 1.

For every basis state $|x\rangle$ appended with n fresh ancillas, applying transversal CNOTs flips the ancilla state to $|x\rangle$. Thus,

$$\Phi_1 = \left(\prod_{i=0}^{n-1} \text{CNOT}_{i,i+n} \right) (\Psi \otimes |0^n\rangle) = \sum_{x=0}^{2^n-1} C_x |\bar{x}\bar{x}\rangle, \quad (2)$$

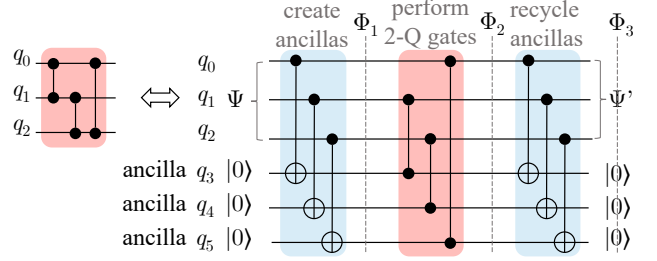


Figure 3: The general case of routing CZs with ancillas. The 3 CZs on the right can be executed simultaneously.

where an overhead line denotes the concatenation of bit-strings. Then, for every pair $(j, j') \in C$, we apply one of the 4 possible CZs,

$$\Phi_2 = \left(\prod_{(j,j') \in C} CZ_{j(+n),j'(+n)} \right) \Phi_1 = \sum_{x=0}^{2^n-1} C_x \prod_{(j,j') \in C} \left((-1)^{\bar{x}\bar{x}_{j(+n)} \cdot \bar{x}\bar{x}_{j'(+n)}} |\bar{x}\bar{x}\rangle \right) = \sum_{x=0}^{2^n-1} C_x \prod_{(j,j') \in C} (-1)^{x_j x_{j'}} |\bar{x}\bar{x}\rangle, \quad (3)$$

where we use the fact that both the j -th bit (from the left) and the $(j+n)$ -th bit of $\bar{x}\bar{x}$ equals the j -th bit of x , similarly for j' . Applying transversal CNOTs, again, flips every state $|\bar{x}\bar{x}\rangle$ back to $|x\rangle|0^n\rangle$, i.e.,

$$\Phi_3 = \left(\prod_{i=0}^{n-1} \text{CNOT}_{i,i+n} \right) \Phi_2 = \Psi' \otimes |0^n\rangle, \quad (4)$$

which finishes our proof.

Note that CZ gates are commutable, so the ones in Eq. 3 can be applied in any order, which may unlock some freedom in scheduling. Moreover, for each $(j, j') \in C$, we have 4 possible CZs to use in Eq. 3 and many of them can be parallelized. For example, in Fig. 3, $n = 3$, and the original CZs are $C = \{(0, 1), (1, 2), (2, 0)\}$ which takes at least 3 steps. Using the procedure just presented, the CZs on $(0+n, 1)$, $(1+n, 2)$, $(2+n, 0)$ can be scheduled to just one step.

3.3 Flying Ancillas in FPQA

The flying ancillas scheme proves particularly advantageous for FPQA over other QC platforms, owing to its high-fidelity movements. The most similar setting is in a multi-chain ion trap QPU [6], where chains of ions are laid out in 1D, and two chains can be moved to merge or split again. However, because there is no distinction between stationary and movable qubits like in FPQA, moving a regular qubit in the ion trap quantum computer has the same cost as moving an ancilla, so the flying ancilla scheme does not hold a big advantage. Additionally, the limited number of qubits available on ion trap QPUs discourages leveraging numerous ancillas. Flying qubits, typically optical, are also employed as communication resources between individual superconducting QPUs but face challenges, including a low interfacing fidelity of approximately 80% per flying qubit [21]. In contrast, in FPQA, the two extra CNOTs required by flying ancilla can achieve 99.5% fidelity, and the ancilla movement has negligible error [13]. Despite this high fidelity, the state-of-the-art FPQA compilation work [32] primarily utilizes only the movement of data qubits for routing, overlooking the potential advantages of routing via flying ancillas.

4 ROUTING FRAMEWORK

4.1 Overview

Given a target problem, the input values to the router are (1) the SLM array parameters (#rows, #columns, and locations), (2) the AOD

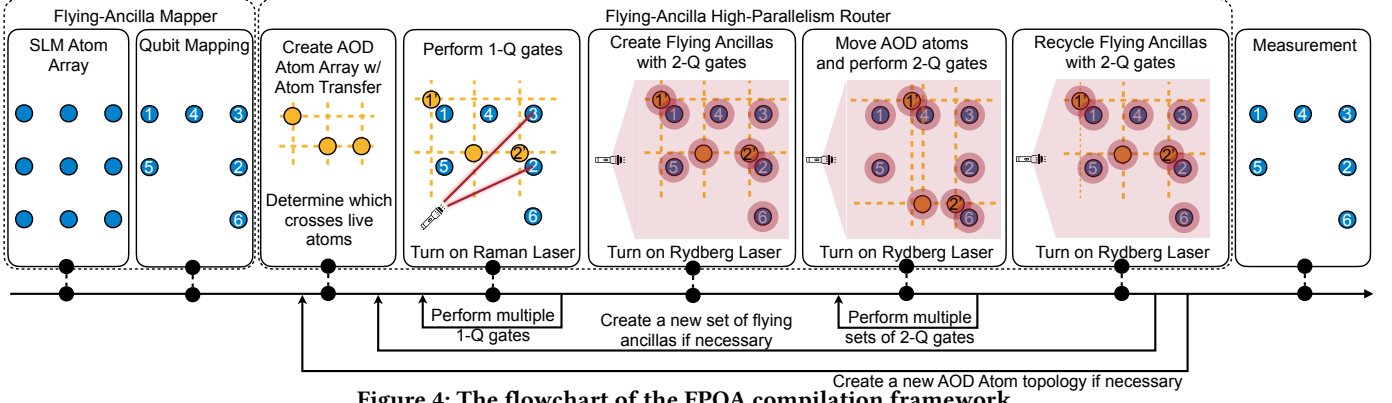


Figure 4: The flowchart of the FPQA compilation framework.

array parameters (#rows and #columns), and (3) the qubit mapping. We focus on routing in this work, so we simply map qubits in reading order throughout. We refer to them as *configurations* of the FPQA. Based on a configuration and the target problem, we leverage a high-parallelism router to generate an optimized schedule. A fast performance evaluator can efficiently return the corresponding performance metric or cost, including the number of 1-Q gates, 2-Q gates, the circuit depth, and movement distance, which are closely related to the circuit fidelity.

With this performance evaluator, our compiler also supports router-in-the-loop FPQA architecture design space exploration. We can use the evaluated cost as feedback to optimize a configuration that targets higher circuit fidelity iteratively. After certain epochs, the compiler will output the best configuration and optimized schedule.

4.2 Compilation of General Quantum Circuit

The compilation process is shown in Fig. 4. Given a quantum circuit, we first decompose the target circuit into 1-Q rotations and 2-Q CZ gates. Then, the gates are performed in alternating 1-Q and 2-Q stages. In the 1-Q stages, we turn on the individual addressable Raman laser to perform the desired gates on the target qubits. After all the available 1-Q gates are done, we move to 2-Q stages. In such stages, we select a set of CZ gates from the non-dependent front-layer of the circuit that can be performed in parallel. Then, we create flying ancillas from the control qubits and move the ancillas close to their corresponding target qubits. We turn on the Rydberg laser so that the ancillas will perform CZ gates with the target qubits. At last, we recycle these ancillas with CNOT gates and repeat this process. After all gates are done, we perform measurements and get results.

4.3 Customized Router for Quantum Simulation

For specific applications, we propose domain-specific routing strategies for higher parallelism. The first application is quantum simulation. To simulate the evolution under a Pauli string, the core part of the simulation algorithm works as follows: First, select a starting qubit inside the given Pauli string and then perform CNOTs on all pairs between the starting one and other qubits in the string.

We propose a longest path-based algorithm to compile this problem on the FPQA, described in Alg. 1. We configure the AOD array so that all ancilla qubits are on the *diagonal* of the grid and can be moved with the best flexibility. Then, we select the qubit i with the smallest index and fan out its state to all AOD ancilla qubits. To maximize the parallelism, we need to find the longest legal path in the dependency graph, where each qubit points to all other qubits in its lower-right corner, as shown in Fig. 5.

Algorithm 1: Customized router for quantum simulation

Data: List P : qubits in the Pauli string with non- I paulis
 $s \leftarrow \emptyset$; Schedule for the compiled program
 $P \leftarrow P \setminus P[0]$; $P[0]$ is the root qubit
 $g \leftarrow \emptyset$; Directed compatibility graph. Two qubits are compatible if and only if there is a path between them.
for $q_i \in P$ **do**
 $g.nodes \leftarrow g.nodes \cup q_i$;
for $q_i \in P$ **do**
 for $q_j \in P \setminus q_i$ **do**
 if $q_j.row \geq q_i.row \ \& \ q_j.col \geq q_i.col$ **then**
 $g.edges \leftarrow g.edges \cup (q_i, q_j)$;
while $g \neq \emptyset$ **do**
 Find the longest path l in g ;
 $s \leftarrow s \cup \text{GenerateSchedule}(l)$;
 $g \leftarrow g \setminus \{n | n \in l\}$;
Result: Schedule s

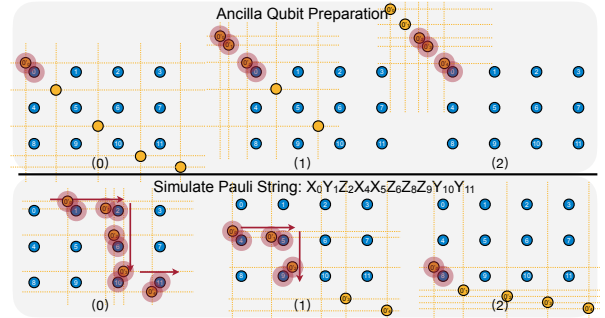


Figure 5: Q-Pilot routing quantum simulation circuits.

Given this longest path, we move the AOD ancilla qubits to their target SLM qubits and perform CNOTs in parallel. Further, those executed qubits will be removed from the candidate set, and the longest path-finding procedure will repeat until all gates are executed. Note that this longest path-finding can be implemented efficiently with dynamic programming. Compared to the generic router, which applies atom transfer to create and recycle ancilla qubits at each stage, this specialized router will maintain the states on the ancilla qubits across stages for one Pauli string, thus having a lower overhead.

4.4 Customized Router for QAOA

Another task that can be highly parallel is Quantum Approximate Optimization Algorithm (QAOA). In QAOA, we are given a graph, e.g., Fig. 6, and our target is to perform 2-Q gates on all its edges.

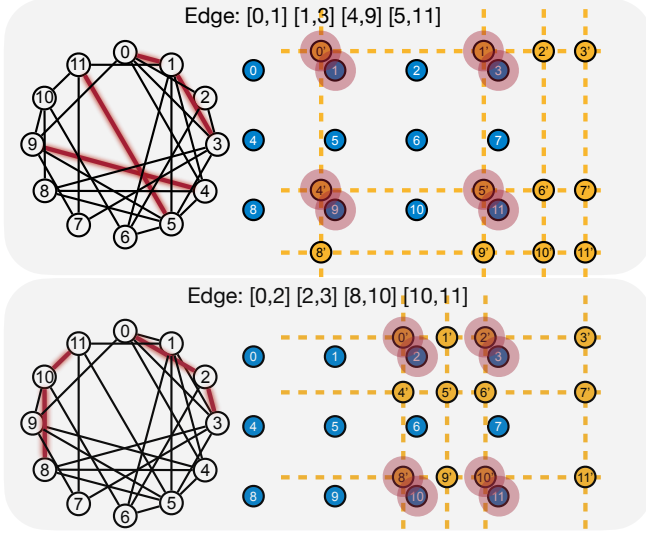


Figure 6: Scheduling of a QAOA circuit using Q-Pilot. The graph representation of the circuit is shown on the left. The edges correspond to interactions between two qubits.

First, we create one ancilla for each qubit. These ancillas will be recycled once the whole graph is done. Then, our router completes this task in a multi-stage way. Each stage will perform one or multiple 2-Q gates corresponding to some edges in the graph. We illustrate the detailed procedure of the first stage in Fig. 6. Among all the qubit pairs, we select the one with the smallest index as the highest-priority pair to begin with, e.g., (0',1). Since each AOD row and column must move simultaneously, we check which pair can be performed inside the same row, e.g., (1',3) is matched in this case. The rest of the AOD columns have already moved outside the SLM array. Then, they will not interact with any SLM atoms. Once the locations of all ancilla qubits in the first row are determined, other ancilla qubits on the rest of the rows can only move vertically due to the grid constraint. Then, we need to determine the vertical location of each row one by one. For the second AOD row, we found the best vertical location that can allow the most pairs to interact, e.g., in this case, we match two pairs (4',9) and (5',11). Note that any undesired interaction is illegal and thus should be avoided. This process will repeat until no rows can legally interact with any SLM atoms. Then, we can determine the locations of all AOD qubits and turn on the Rydberg laser to perform the parallel 2-Q gates. This greedy algorithm always tries to achieve the maximum matching on the first row and ultimately reaches a schedule with max parallelism.

So far, we have finished the first stage of the schedule with four 2-Q gates being performed. In the second stage, the highest-priority pair now becomes (0',2), and the same procedure as stage one can be applied to find a legal schedule with maximal parallelism. After t stages, the compilation flow ends with a t -stage legal schedule where all 2-Q gates are performed. Lastly, we recycle the ancillas and complete the task.

5 EVALUATION

5.1 Evaluation Methodology

Benchmarks. We utilize three benchmark types: random, quantum simulation, and QAOA circuits. Benchmarks were created for 5, 10, 20, 50, and 100 qubits. Random circuits were generated with Qiskit's `random_circuit` function, which randomly places 1-Q and 2-Q gates on qubits. The number of CNOT gates is set at 2x, 5x, 10x, 20x, and

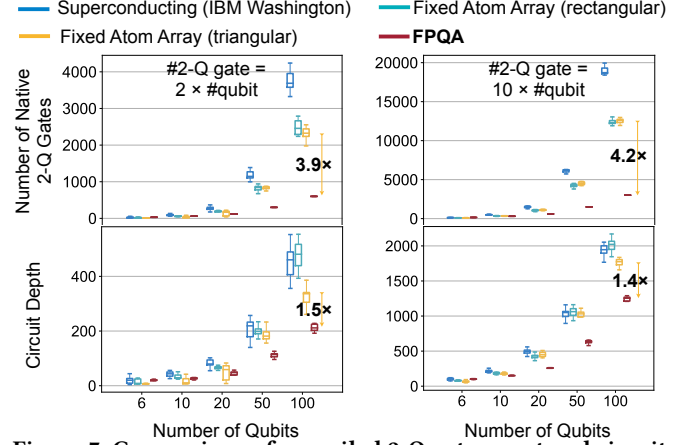


Figure 7: Comparison of compiled 2-Q gate count and circuit depth between Q-Pilot and the three baselines on random circuits. The random circuits vary in size, from 5-Q to 100-Q, and have 2-Q gate count between $2\times$ and $10\times$ qubit count.

50x the qubit count. Quantum simulation circuits were formed from 100 random Pauli strings. The probability p of a qubit having a Pauli operator X , Y , or Z varies from 0.1 to 0.5. QAOA circuits were constructed using ZZ gates between random qubit pairs. These pairs had an edge probability p of 0.1 to 0.5. We also designed specific QAOA circuits based on 3-regular and 4-regular graphs. These circuits also used 5, 10, 20, 50, and 100 qubits.

Baselines. 3 devices as baselines were chosen: the 127-qubit IBM Washington machine, a 16×16 square lattice, and a 16×16 triangular lattice of fixed neutral atoms, following Ref. [32]. The IBM machine features a heavy hexagon coupling graph. The square lattice's atoms connect to four nearest neighbors, while the triangular lattice's atoms connect to six. Qiskit's transpiler compiled benchmark circuits onto these devices at optimization level 3. Circuit depth, defined as the number of parallel 2-Q gate layers, was a key comparison metric, alongside the number of 2-Q gates in each *compiled* circuit for the baseline devices and Q-Pilot. Additionally, Q-Pilot was benchmarked against the solver-based compiler from Ref. [32], used for QAOA problems on 3- and 4-regular graphs. Comparisons included circuit depths and compilation times, with a 4,000s timeout ($\sim$ an hour) set for the solver-based compiler due to its exponential runtime scaling.

5.2 Main Results

Results on random circuits. Fig. 7 shows the results of compiling random circuits. Compared with three baseline devices, for 100 qubits Q-Pilot shows an average of $4.2\times$ reduction in the compiled 2-Q gate count, as well as an average of $1.4\times$ reduction in compiled circuit depth compared with the best-performing baseline approach.

Results on quantum simulation circuits. Fig. 8 shows the results of compiling quantum simulation circuits. For Pauli probabilities 50%, Q-Pilot shows an average of $6.9\times$ reduction in the compiled 2-Q gate count and an average of $27.7\times$ reduction in compiled circuit depth compared with the best-performing baseline on 100-qubit circuits compared with the best baseline. Besides the random Pauli strings, we also test with the Pauli strings used in some molecule simulation problems [19]. As shown in Table 1, Q-Pilot shows an average $1.36\times$ reduction in the 2-Q gate count and average $2.60\times$ circuit depth reduction over the best baseline.

Results on QAOA circuits. Fig. 9 shows the results of compiling Max-Cut QAOA circuits for 4-regular graphs and random graph with

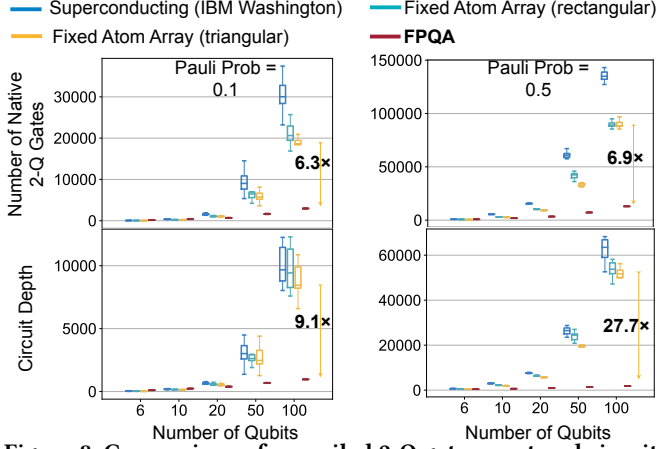


Figure 8: Comparison of compiled 2-Q gate count and circuit depth between Q-Pilot and the three baselines on quantum simulation circuits from 5-Q to 100-Q. The circuits are generated with Pauli probability $p = 0.1$ and 0.5 .

Table 1: Quantum Simulation for Molecule Pauli strings.

Benchmark	Device	H2	LiH_UCCSD	H2O	BeH2
Depth	FAA(rectangular)	76	2,772	31,087	43,919
	FAA(triangular)	61	2,052	26,189	37,314
	Superconducting	77	3,403	40,080	59,259
	Ours	61	849	7,585	10,617
#2Q Gate	FAA(rectangular)	82	3,577	41,306	58,720
	FAA(triangular)	73	2,616	35,353	51,699
	Superconducting	85	5,082	67,247	103,594
	Ours	94	2,130	20,966	29,518

edge occupancy 30%. Q-Pilot again shows an average of 10.0 $\times$ reduction in compiled 2-Q gate count and an average of 6.7 $\times$ reduction in the compiled circuit depth.

Comparison with the Solver-Based Compiler. As illustrated in Table 2, we compare Q-Pilot against the solver-based methods [32, 33] in compiling QAOA circuits for regular graphs. Ref. [33] relaxes the formulation of Ref. [32] to tradeoff compilation time and quality. While these methods achieve better solutions, they struggle with larger problems, often failing to find a solution within an hour due to its exponential runtime scaling. In contrast, Q-Pilot efficiently compiles all these problems in under 1 second, with the compiled circuit depth not exceeding 4 $\times$ the optimal depth.

5.3 Analysis

Impact of Array Size on Circuit Depth. Fig. 10 shows how array sizes affect the compiled circuit depth. We organized the qubits into rectangular arrays of varying widths (8, 16, 32, 64, 128), with the optimal array widths marked by stars in the figure. Optimal array widths vary across different problems, highlighting a trade-off between greater parallelism within a row and across different rows. Specifically, in Fig. 10, we observe that QAOA circuits achieve optimal performance with large array width (128), while random circuits and quantum simulation problems are best served with moderate array widths (64 or 32 in our study). The insight here is while larger array widths offer more parallel execution paths, they might not always correspond to increased efficiency for all types of problems, possibly due to overheads or specific characteristics of the circuit structure.

How does the 2-Q gate error rate affect the overall error rate? Fig. 11 (a) shows the relation between the overall error rate and the 2-Q gate error rate. We model the circuit error with the equation

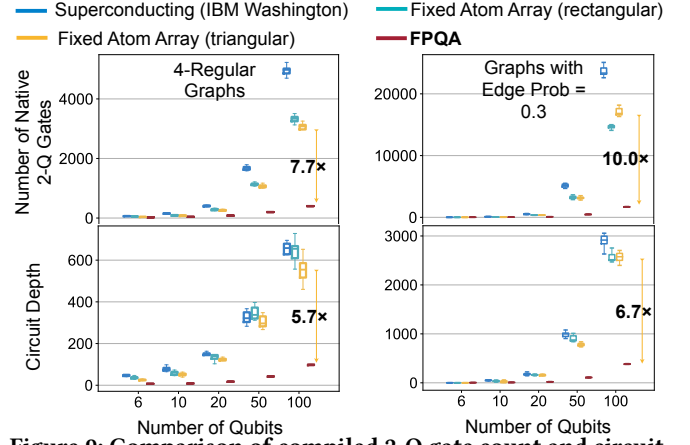


Figure 9: Comparison of compiled 2-Q gate count and circuit depth between Q-Pilot and the three baselines on QAOA circuits. The QAOA circuits vary in size, from 6-Q to 100-Q, and are generated with edge $p = 0.3$ and 4-regular graphs.

Table 2: Comparison of Q-Pilot with solver based method.

Benchmark		6Q	10Q	20Q	50Q	100Q
3-reg.	runtime(s)	solver [32]	0.173	0.381	74.5	timeout
		iter-p [33]	0.509	2.16	14.6	966
		Ours	5.57E-3	9.89E-3	1.07E-2	7.52E-2
	depth	solver [32]	3	3	3	-
		iter-p [33]	3	5	6	10
		Ours	5	7	11	24
4-reg.	runtime(s)	solver [32]	18.1	3.93E3	timeout	timeout
		iter-p [33]	0.852	2.64	23.4	2.34E3
		Ours	6.25E-3	9.31E-3	2.10E-2	7.23E-2
	depth	solver [32]	5	5	-	-
		iter-p [33]	5	6	8	15
		Ours	6	9	15	32

introduced [32], where ϵ is the overall error rate:

$$\epsilon = 1 - f_2^{NT} f_1^{G_1} \exp\left(-N \frac{\sum_i T_0 \sqrt{D_i}}{T_2}\right), \quad (5)$$

N is the maximum number of qubits used (including AOD and SLM), and T is the circuit depth. G_1 is the number of 1-Q gates. f_1 and f_2 are the fidelity of 1-Q and 2-Q gates, respectively. T_2 is the coherence time of the qubit, and T_0 is the characteristic time of atom movement. D_i is the maximum distance atoms moved in stage i . In our estimation, we choose $f_1 = 99.9\%$, $T_2 = 1.5s$, and $T_0 = 300\mu s$ [32]. The three benchmarks used here are 1) quantum simulation circuits with 5 qubits and 100 non-trivial Pauli strings with $p = 0.1$, 2) random 5Q circuits with an average of two 2-Q gates per qubit, and 3) QAOA circuits for random 3-regular graphs. The error rates are below 0.5 when the 2-Q gate has an error rate below 10^{-3} .

What is the distribution of the parallelism? Fig. 11 (b) shows the percentage of stages with the number of 2-Q gates simultaneously executed for QAOA problems. The average parallelism of 20Q, 50Q, and 100Q problems are 3.32, 4.13, and 4.90, respectively. As the problem scales up, the parallelism of the problem is also increased.

Whether the application-specific compilers bring better performance for quantum simulation and QAOA? Fig. 12 shows the advantage of the domain-specific compiler compared to the general compiler. For quantum simulation, the domain-specific compiler reduces the 2-Q gate count by 1.5 $\times$ and the circuit depth by 8.8 $\times$. For QAOA, the domain-specific compiler reduces the 2-Q gate count by 2.8 $\times$ and the circuit depth by 10.1 $\times$. The advantages come from domain-specific heuristics that minimize the circuit depth.

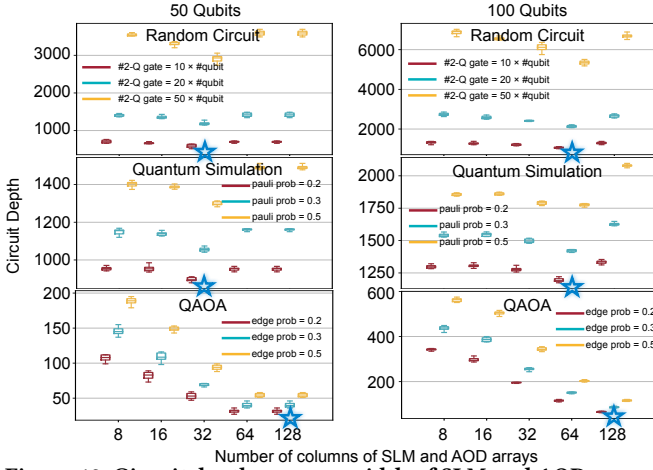


Figure 10: Circuit depth vs array width of SLM and AOD arrays in FPQA. The three benchmark circuits are shown here for 50 qubits and 100 qubits. The star in each graph marks the optimal array width for the smallest circuit depth.

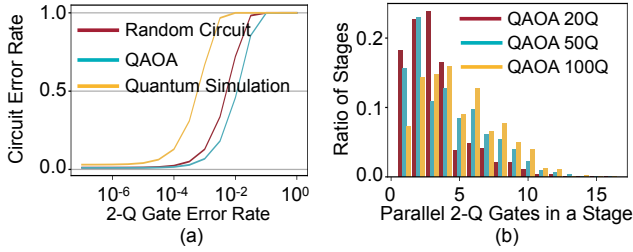


Figure 11: (a) Overall error rate vs. 2-Q gate error rate for random 6Q circuits with two 2Q gates per qubit, QAOA circuits based on random 3-regular graphs, and 5Q quantum simulation circuits with 100 Pauli strings and $p = 0.1$. (b) Ratio of total stages vs number of parallel 2-Q gates in a stage using Q-Pilot on QAOA circuits with 20-Q, 50-Q, and 100-Q.

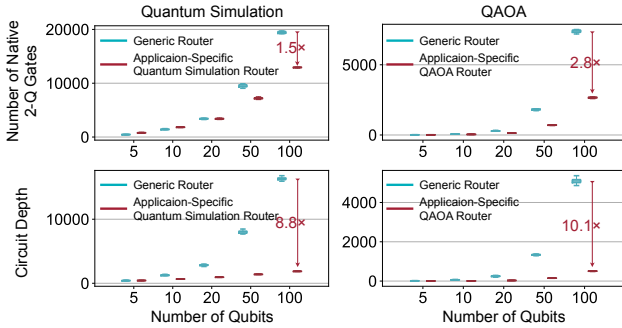


Figure 12: Advantage of our application specific Quantum Simulation and QAOA router comparing to the generic router.

How scalable is the Q-Pilot? We test Q-Pilot with a large number of qubits to show its scalability. For the QAOA problem, we choose random graphs with edge $p = 0.5$. It takes 1.51s, 10.75s, and 129.50s to compile 500, 1000, and 2000 qubits. For quantum simulation problems, we choose 100 random Pauli strings. It takes 6.91s, 14.28s, and 30.48s to compile 500, 1000, and 2000 qubits. We generate random circuits with a depth of 10 for general circuits, and it takes 2.64s, 8.70s, and 32.31s to compile 500, 1000, and 2000 qubits. The fast speed proves that Q-Pilot is scalable and can handle large-scale problems.

6 CONCLUSION AND OUTLOOK

We design a compilation framework for FPQA with movable atoms, enabling flexible qubit mapping and efficient 2-Q gate execution. Our approach includes a versatile router for enhanced parallelism in quantum simulations and QAOA. Future directions involve refining search heuristics for better solutions, exploring quantum error correction in FPQA-based QPUs, and using compiler insights for hardware advancements like multi-functional FPQA zones.

ACKNOWLEDGEMENT

This work is partially supported by NSF grant 442511-CJ-22291, MIT-IBM Watson AI Lab, and Qualcomm Innovation Fellowship. The authors would like to thank Dolev Bluvstein, Mikhail D. Lukin, and Hengyun Zhou for valuable discussions on neutral atom arrays.

REFERENCES

- [1] <https://newsroom.ibm.com/2022-11-09-IBM-Unveils-400-Qubit-Plus-Quantum-Processor-and-Next-Generation-IBM-Quantum-System-Two>.
- [2] <https://www.rigetti.com/>.
- [3] <https://ai.googleblog.com/2018/03/a-preview-of-bristlecone-googles-new.html>.
- [4] <https://spectrum.ieee.org/tech-talk/computing/hardware/intels-49qubit-chip-aims-for-quantum-supremacy>.
- [5] <https://www.quera.com/aquila>.
- [6] <https://ionq.com/posts/august-25-2021-deep-dive-reconfigurable-multicore-quantum-architecture>.
- [7] M. Alam et al. An efficient circuit compilation flow for quantum approximate optimization algorithm. DAC'20.
- [8] M. Alam et al. Noise resilient compilation policies for quantum approximate optimization algorithm. ICCAD'20.
- [9] J. M. Baker et al. Exploiting long-distance interactions and tolerating atom loss in neutral atom quantum architectures. ISCA'21.
- [10] D. Bluvstein et al. A quantum processor based on coherent transport of entangled atom arrays. *Nature* 604, 7906 (2022), 451–456.
- [11] S. Brandthofer et al. Optimal mapping for near-term quantum architectures based on Rydberg atoms. ICCAD'21.
- [12] S. Ebadi et al. Quantum optimization of maximum independent set using Rydberg atom arrays. *Science* 376, 6598 (2022), 1209–1215.
- [13] S. J. Evered et al. High-fidelity parallel entangling gates on a neutral atom quantum computer. *Nature* 622 (2023), 268–272.
- [14] H. Fan et al. Optimizing quantum circuit placement via machine learning. DAC'22.
- [15] P. Gokhale et al. Quantum fan-out: circuit optimizations and technology. QCE'21.
- [16] T. M. Graham et al. Multi-qubit entanglement and algorithms on a neutral-atom quantum computer. *Nature* 604, 7906 (2022), 457–462.
- [17] P. Høyer et al. Quantum fan-out is powerful. *Theory of Computing* 1, 5 (2005).
- [18] G. Li et al. Tackling the qubit mapping problem for NISQ-era quantum. ASPLOS'19.
- [19] G. Li et al. Paulihedral: a generalized block-wise compiler optimization framework for quantum simulation kernels. ASPLOS'22.
- [20] Y. Li et al. Timing-aware qubit mapping and gate scheduling adapted to neutral atom quantum computing. TCAD 42, 11 (2023).
- [21] P. Magnard et al. Microwave quantum link between superconducting circuits housed in spatially separated cryogenic systems. *PRL* 125 (2020), 260502.
- [22] D. Maslov et al. Quantum Circuit Placement. TCAD 27, 4 (2008), 752–763.
- [23] Abtin Molavi et al. Qubit mapping and routing via MaxSAT. MICRO'22.
- [24] P. Murali et al. Full-stack, real-system quantum computer studies: architectural comparisons and design insights. ISCA'19.
- [25] M. A. Norcia et al. Iterative assembly of ^{171}Yb atom arrays in cavity-enhanced optical lattices. arXiv:2401.16177.
- [26] S. Park et al. A fast and scalable qubit-mapping method for noisy intermediate-scale quantum computers. DAC'22.
- [27] T. Patel et al. Geyser: a compilation framework for quantum computing with neutral atoms. ISCA '22.
- [28] M. Saffman. Quantum computing with atomic qubits and Rydberg interactions: progress and challenges. *Journal of Physics B* 49, 20 (oct 2016), 202001.
- [29] M. Y. Siraichi et al. Qubit allocation. CGO'18.
- [30] B. Tan et al. Optimal layout synthesis for quantum computing. ICCAD'20.
- [31] B. Tan et al. Optimal qubit mapping with simultaneous gate absorption. ICCAD'21.
- [32] B. Tan et al. Qubit mapping for reconfigurable atom arrays. (2022). ICCAD'22.
- [33] D. B. Tan et al. Compiling quantum circuits for dynamically field-programmable neutral atoms array processors. *Quantum* 8 (2024).
- [34] R. Wille et al. Mapping quantum circuits to IBM QX architectures using the minimal number of SWAP and H operations. DAC'19.
- [35] T.-A. Wu et al. A robust quantum layout synthesis algorithm with a qubit mapping checker. ICCAD'22.
- [36] X. Zhou et al. A Monte Carlo tree search framework for quantum circuit transformation. ICCAD'20.
- [37] A. Zulehner et al. Efficient mapping of quantum circuits to the IBM QX architectures. DATE'18.